PALIANDROM USING MASM

```
.model small
printmsg macro msg
push ax
push dx
LEA dx,msg
mov ah,09h
int 21h
pop dx
pop ax
endm
.data
	msg1 db 0ah,0dh,"enter the string$"
	msg2 db 0ah,0dh,"it is a palindrome$"
	msg3 db 0ah,0dh,"it is not a palindrome$"
	msg4 db 0ah,0dh,"$"
	msg5 db 0ah,0dh,"length is$"
	buffer db 10
	db 12 dup("$")
	len db 00
.code
	mov ax,@data
	mov ds,ax
	printmsg msg1
	LEA dx ,buffer
	mov ah,0ah
	int 21h
	printmsg msg4
	mov si,02
	LEA dx,buffer[si]
	mov ah,09h
	int 21h
	mov al,buffer[01]
	mov len,al
	printmsg msg5
	mov dl,len
	add dl,30h
	mov ah,02h
	int 21h
	mov si,02
	mov al,len
	XOR ah,ah
	mov di,ax
	mov cl,00
	a1:cmp cl,len
	je a4
	mov dl,buffer[si]
	mov dh,buffer[di+1]
	cmp dl,dh
	jnz a2
	inc si
	dec di
```

```
inc cl
jmp a1
a4:printmsg msg2
jmp a3
a2:printmsg msg3
a3:mov ah,4ch
int 21h
end
```

Output

```
enter the string    malayalam
malayalam
length is9
it is a palindrome
```

FIBONACCI SERIES USING MASM

```
.model small
.stack 100h

printmsg macro msg
    push ax
    push dx
    lea dx, msg
    mov ah, 09h
    int 21h
    pop dx
    pop ax
endm

.data
msg1 db "Enter the limit:$"
msg2 db 0ah,0dh,"The series is:$"
msg3 db " $"
num dw 0
cnt dw 0

.code
mov ax, @data
mov ds, ax

printmsg msg1
call readnum
mov cx, num
cmp cx, 0
je exit

printmsg msg2
mov num, 0
call printnum
dec cx
cmp cx, 0
je exit

printmsg msg3
mov num, 1
call printnum
dec cx

mov ax, 0
mov bx, 1

fib_loop:
    cmp cx, 0
    je exit
    printmsg msg3
    add ax, bx
```

```
    mov num, ax
    call printnum
    mov dx, ax
    mov ax, bx
    mov bx, dx
    dec cx
    jmp fib_loop

exit:
mov ah, 4Ch
int 21h

readnum proc near
    push ax
    push bx
    push cx
    push dx
    mov num, 0
r1:
    mov ah, 01h
    int 21h
    cmp al, 0dh
    je r2
    mov cx, ax
    and cx, 00ffh
    sub cx, 30h
    mov bx, 10
    mov ax, num
    mul bx
    add ax, cx
    mov num, ax
    jmp r1
r2:
    pop dx
    pop cx
    pop bx
    pop ax
    ret
readnum endp

printnum proc near
    push ax
    push bx
    push cx
    push dx
    mov ax, num
    mov bx, 10
p1:
    mov dx, 0
    cmp ax, 0
    jz p2
    div bx
    push dx
    inc cnt
```

```
    jmp p1
p2:
    cmp cnt, 0
    je p3
    pop dx
    add dl, 30h
    mov ah, 02h
    int 21h
    dec cnt
    jmp p2
p3:
    pop dx
    pop cx
    pop bx
    pop ax
    ret
printnum endp

end
```

Output

```
Enter the limit:5
The series is:0 1 1 2 3
```

CALCULATOR USING MASM

```
.model small
.stack 100h

printmsg macro msg
    push ax
    push dx
    lea dx, msg
    mov ah, 09h
    int 21h
    pop dx
    pop ax
endm

.data
msg1 db 0ah,0dh,"Enter the first number:$"
msg2 db 0ah,0dh,"Enter the second number:$"
msg3 db 0ah,0dh,"1-Add, 2-Sub, 3-Mul, 4-Div:$"
msg4 db 0ah,0dh,"Your result is:$"
msg5 db 0ah,0dh,"NO SUCH OPTION$"
num dw 0
cnt dw 0
choice dw 0
a dw 0
b dw 0

.code
mov ax, @data
mov ds, ax

printmsg msg1
call readnum
mov ax, num
mov a, ax

printmsg msg2
call readnum
mov ax, num
mov b, ax

printmsg msg3
call readnum
mov ax, num
mov choice, ax

cmp choice, 1
je addi
cmp choice, 2
je subt
cmp choice, 3
je multi
cmp choice, 4
```

```
je divi
printmsg msg5
jmp exit

addi:
   printmsg msg4
   mov ax, a
   add ax, b
   mov num, ax
   call printnum
   jmp exit

subt:
   printmsg msg4
   mov ax, a
   sub ax, b
   mov num, ax
   call printnum
   jmp exit

multi:
   printmsg msg4
   mov ax, a
   mov bx, b
   mul bx
   mov num, ax
   call printnum
   jmp exit

divi:
   printmsg msg4
   mov ax, a
   mov bx, b
   xor dx, dx
   div bx
   mov num, ax
   call printnum
   jmp exit

exit:
mov ah, 4Ch
int 21h

readnum proc near
   push ax
   push bx
   push cx
   push dx
   mov num, 0
r1:
   mov ah, 01h
   int 21h
   cmp al, 0dh
   je r2
```

```
    mov cx, ax
    and cx, 00ffh
    sub cx, 30h
    mov bx, 10
    mov ax, num
    mul bx
    add ax, cx
    mov num, ax
    jmp r1
r2:
    pop dx
    pop cx
    pop bx
    pop ax
    ret
readnum endp

printnum proc near
    push ax
    push bx
    push cx
    push dx
    mov ax, num
    mov bx, 10
p1:
    mov dx, 0
    cmp ax, 0
    jz p2
    div bx
    push dx
    inc cnt
    jmp p1
p2:
    cmp cnt, 0
    je p3
    pop dx
    add dl, 30h
    mov ah, 02h
    int 21h
    dec cnt
    jmp p2
p3:
    pop dx
    pop cx
    pop bx
    pop ax
    ret
printnum endp

end
```

Output

```
Enter the first number:12
Enter the second number:4
1-Add, 2-Sub, 3-Mul, 4-Div:1
Your result is:16

Enter the first number:12
Enter the second number:4
1-Add, 2-Sub, 3-Mul, 4-Div:2
Your result is:8

Enter the first number:12
Enter the second number:4
1-Add, 2-Sub, 3-Mul, 4-Div:3
Your result is:48

Enter the first number:12
Enter the second number:4
1-Add, 2-Sub, 3-Mul, 4-Div:4
Your result is:3
```

STRING REVERSE

```
.model small
.stack 100h

printmsg macro msg
    push ax
    push dx
    lea dx, msg
    mov ah, 09h
    int 21h
    pop dx
    pop ax
endm

.data
    msg1 db 0ah, 0dh, "Enter the string:$"
    msg2 db 0ah, 0dh, "Reversed string is:$"
    msg3 db 0ah, 0dh, "String is:$"
    msg4 db 0ah, 0dh, "$"
    msg5 db 0ah, 0dh, "Length is $"

    buffer db 10                    ; Max input length
           db ?                     ; Actual length
           db 10 dup(?)             ; Input characters

    revStr db 12 dup('$')           ; Reversed string
    len db 0

.code
main:
    mov ax, @data
    mov ds, ax

    ; Ask for input
    printmsg msg1

    lea dx, buffer
    mov ah, 0Ah
    int 21h                         ; input string

    printmsg msg4

    ; Show original string
    printmsg msg3
    lea dx, buffer + 2
    mov ah, 09h
    int 21h

    ; Get the length
    mov al, buffer[1]
    mov len, al

    ; Show length
```

```
    printmsg msg5
    mov dl, len
    add dl, 30h
    mov ah, 02h
    int 21h

    ; Reverse logic
    mov cl, 0           ; counter for loop
    mov si, 2           ; start of input string
    mov di, 0           ; index for revStr

reverse_loop:
    cmp cl, len
    je done_reversing

    mov al, len
    sub al, cl
    dec al
    xor ah, ah
    add ax, si          ; ax = si + len - cl - 1
    mov bx, ax

    mov dl, buffer[bx]
    mov revStr[di], dl

    inc cl
    inc di
    jmp reverse_loop

done_reversing:
    mov revStr[di], '$'     ; null terminate reversed string

    ; Display reversed string
    printmsg msg2
    lea dx, revStr
    mov ah, 09h
    int 21h

    ; Exit program
    mov ah, 4Ch
    int 21h

end main
```

Output

```
Enter the string:
hello
String is:hello
Length is 5
Reversed string is:olleh
```

Implementation of simple Decimal Arithmetic And Bit manipulation (direct value assigning) using Kit

1.Addition

starting address : 1000
mov ax,05
mov bx,02
add ax,bx
mov [1500],ax
Hlt
Int

Go 1000
Int
Sw 1500

2.Substraction

starting address : 2000
mov ax,05
mov bx,02
sub ax,bx
mov [2500],ax
Hlt
Int

Go 2000
Int
Sw 2500

Multiplication

starting address : 3000
mov ax,05
mov bx,02
mul bx
mov [3500],ax
Hlt
Int

Go 3000
Int
Sw 3500

Division

```
starting address : 4000
mov ax,05
mov bx,02
Xor dx,dx
div bx
mov [4500],ax
mov [5000],dx
Hlt
Int


Go 4000
Int
Sw 4500
int
Sw 5000
```

Implementation of simple Decimal Arithmetic And Bit manipulation value assigned by address location using Kit

1.Addition

```
starting address : 1000
mov ax,[1200]
mov bx,[1202]
add ax,bx
mov [1500],ax
Hlt
Int

Sw 1200
0004                    //enter the value to ax
Int

Sw 1202
0002                    // value to bx

Go 1000
Int
Sw 1500
```

2.Substraction

```
starting address : 2000
mov ax,[2200]
mov bx,[2202]
sub ax,bx
mov [2500],ax
Hlt
Int

Sw 2200
0004                    //enter the value to ax
Int

Sw 2202
0002                    // value to bx



Go 2000
Int
Sw 2500
```

Multiplication

```
starting address : 3000
mov ax,[3200]
mov bx,[3202]
mul bx
mov [3500],ax
Hlt
Int

Sw 3200
0004                    //enter the value to ax
Int

Sw 3202
0002                    // value to bx


Go 3000
Int
Sw 3500
```

Division

```
starting address : 4000
mov ax,[4200]
mov bx,[4202]
Xor dx,dx
div bx
mov [4500],ax
mov [5000],dx
Hlt
Int

Sw 4200
0004                    //enter the value to ax
Int

Sw 4202
0002                    // value to bx



Go 4000
Int
Sw 4500
Sw 5000
```

Implementation of simple Decimal Arithmetic And Bit manipulation using Kit (Menu Driven)

```
1000  MOV AX, [2000]
      MOV BX, [2002]
      MOV CX, [2004]
      CMP CX, 01
      JZ 1024
      CMP CX, 02
      JZ 102B
      CMP CX, 03
      JZ 1032
      CMP CX, 04
       JZ 1039

1024  ADD AX, BX
      MOV [3000], AX
      HLT

102B  SUB AX, BX
      MOV [3000], AX
      HLT

1032  MUL BX
      MOV [3000], AX
      HLT

1039  XOR DX, DX
      DIV BX
      MOV [3000], AX
      HLT
```

Stepper motor Anti Clockwise 360 degree

Starting adress:1000

```
MOV AL, 80H
OUT 56, AL

MOV CX, 0032H


   MOV AL, 09H
   OUT 50, AL
   CALL 2000

   MOV AL, 05H
   OUT 50, AL
   CALL 2000

   MOV AL, 06H
   OUT 50, AL
   CALL 2000

   MOV AL, 0AH
   OUT 50, AL
   CALL 2000

   DEC CX
   CMP CX, 0000H
   JNZ 1009

CALL 2000
HLT
```

Starting adress:2000

```
   MOV DX, 1000H
   DEC DX
   CMP DX, 0000H
   JNZ 2004
   RET
```

Stepper motor Clockwise 180 degree

Starting adress:1000

```
MOV AL, 80H
OUT 56, AL

MOV CX, 0019H


    MOV AL, 0AH
    OUT 50, AL
    CALL 2000

    MOV AL, 06H
    OUT 50, AL
    CALL 2000

    MOV AL, 05H
    OUT 50, AL
    CALL 2000

    MOV AL, 09H
    OUT 50, AL
    CALL 2000

    DEC CX
    CMP CX, 0000H
    JNZ 1009

CALL 2000
HLT
```

Starting adress:2000

```
    MOV DX, 1000H
    DEC DX
    CMP DX, 0000H
    JNZ 2004
    RET
```

DAC -SQUAR WAVE

```
2000: MOV AL, 80H
2003: OUT 56, AL

2005: MOV AL, 00H
2008: OUT 50, AL
200A: CALL 4000

200D: MOV AL, 0FFH
200F: OUT 50, AL
      CALL 4000

      JMP 2005
      HLT
      INT

4000: MOV CX, 100H
4004: LOOP 4004
      RET
```

DAC -TRIAGULAR WAVE

```
1000: MOV AL, 80H
1003: OUT 56, AL

1005: MOV AL, 00H
1008: OUT 50, AL

100A: INC AL
100B: CMP AL, 0FFH
100E: JNE 1008

1011: OUT 50, AL
1013: DEC AL
1015: CMP AL, 00H
1018: JNE 1011
101A: JMP 1006

      HLT
```

DAC - SAWTOOTH WAVE

```
2000: MOV AL, 80H
2000: OUT 56, AL

2005: MOV AL, 00H
2008: OUT 50, AL

200A: MOV AL, 0FFH
200D: OUT 50, AL
200F: DEC AL
2011: CMP AL, 00H
2014: JNE 200D
2016: JMP 2005

        HLT
```

PASS ONE

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

int main() {
    FILE *f1,*f2,*f3,*f4,*f5;
    int lc,sa,op1,len,o;
    char la[20],m1[20],op[20],otp[20];

    // Open files
    f1=fopen("input.txt","r");   // Source program
    f2=fopen("optab.txt","r");   // Optable (mnemonic + size)
    f3=fopen("symtab.txt","w");  // Symbol table
    f4=fopen("interm.txt","w");  // Intermediate file
    f5=fopen("length.txt","w");  // Program length

    // Read first line
    fscanf(f1,"%s%s%d",la,m1,&op1);
    if(strcmp(m1,"START")==0) {
        sa=op1;
        lc=sa;
        fprintf(f4,"%d\t%s\t%s\t%d\n",lc,la,m1,op1);
    } else {
        lc=0;
    }

    // Read next line (loop begins)
    fscanf(f1,"%s%s%s",la,m1,op);
    while(strcmp(m1,"END")!=0) {
        fprintf(f4,"%d\t%s\t%s\t%s\n",lc,la,m1,op);
```

```
        // If label exists → add to symbol table
        if(strcmp(la,"-")!=0) {
            fprintf(f3,"%d\t%s\n",lc,la);
        }

        // Check if mnemonic is in optab
        rewind(f2);
        int found=0;
        while(fscanf(f2,"%s%d",otp,&o)!=EOF) {
            if(strcmp(m1,otp)==0) {
                lc+=o;   // increment LC by instruction size
                found=1;
                break;
            }
        }

        // If not found in optab, check directives
        if(!found) {
            if(strcmp(m1,"WORD")==0) {
                lc+=3;
            } else if(strcmp(m1,"RESW")==0) {
                lc += 3*atoi(op);
            } else if(strcmp(m1,"BYTE")==0) {
                if(op[0]=='X') lc+=1;
                else lc+=strlen(op)-3; // e.g., C'Z' → 1 byte
            } else if(strcmp(m1,"RESB")==0) {
                lc += atoi(op);
            }
        }

        fscanf(f1,"%s%s%s",la,m1,op);
    }

    // Write last END line into interm file
    fprintf(f4,"%d\t%s\t%s\t%s\n",lc,la,m1,op);

    // Program length
    fprintf(f5,"%d",lc-sa);

    fclose(f1);
    fclose(f2);
    fclose(f3);
    fclose(f4);
    fclose(f5);
    return 0;
}
```

```
saabith_kolakkattil@sk-VirtualBox:~$ cat input.txt
COPY START 1000
-    LDA   FIVE
-    STA   ALPHA
-    LDCH  CHARZ
-    STCH  C1
FIVE  WORD  5
```

```
ALPHA RESW  1
CHARZ BYTE  C'Z'
C1    RESB  1
-     END   COPY

saabith_kolakkattil@sk-VirtualBox:~$ cat optab.txt
LDA     3
STA     3
LDCH    3
STCH    3

saabith_kolakkattil@sk-VirtualBox:~$ cat interm.txt
1000    COPY    START   1000
1000    -       LDA     FIVE
1003    -       STA     ALPHA
1006    -       LDCH    CHARZ
1009    -       STCH    C1
1012    FIVE    WORD    5
1015    ALPHA   RESW    1
1018    CHARZ   BYTE    C'Z'
1019    C1      RESB    1
1020    -       END     COPY
saabith_kolakkattil@sk-VirtualBox:~$ cat symtab.txt
1012    FIVE
1015    ALPHA
1018    CHARZ
1019    C1
saabith_kolakkattil@sk-VirtualBox:~$ cat length.txt
20
```

Pass two

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

int main() {
    FILE *fin, *fsym, *fopt, *flen;
    char label[20], opcode[20], operand[20];
    char optab_mne[20], optab_code[20], sym_name[20];
    int loc, start_addr, prog_len, operand_addr;

    fin = fopen("interm.txt","r");
    fsym = fopen("symtab.txt","r");
    fopt = fopen("optab.txt","r");
    flen = fopen("length.txt","r");

    fscanf(flen,"%d",&prog_len);
    fscanf(fin,"%d%s%s%d",&loc,label,opcode,&start_addr);
    printf("H^%s^%06d^%06d\n",label,start_addr,prog_len);

    fscanf(fin,"%d%s%s%s",&loc,label,opcode,operand);
    printf("T^%06d^",loc);

    while(strcmp(opcode,"END")!=0) {
        rewind(fopt);
        int found=0;
        while(fscanf(fopt,"%s%s",optab_mne,optab_code)!=EOF) {
            if(strcmp(opcode,optab_mne)==0) {
                found=1;
                break;
            }
```

```
        }

        if(found) {
            rewind(fsym);
            while(fscanf(fsym,"%d%s",&operand_addr,sym_name)!=EOF) {
                if(strcmp(operand,sym_name)==0) {
                    break;
                }
            }
            printf("%s%04d^",optab_code,operand_addr);
        }
        else if(strcmp(opcode,"WORD")==0) {
            printf("%06d^",atoi(operand));
        }
        else if(strcmp(opcode,"BYTE")==0) {
            if(operand[0]=='C') {
                char c = operand[2];
                printf("%02X^",c);
            }
            else if(operand[0]=='X') {
                char hex[3];
                hex[0]=operand[2];
                hex[1]=operand[3];
                hex[2]='\0';
                printf("%s^",hex);
            }
        }
        else if(strcmp(opcode,"RESW")==0 || strcmp(opcode,"RESB")==0) {
        }

        fscanf(fin,"%d%s%s%s",&loc,label,opcode,operand);
    }

    printf("\nE^%06d\n",start_addr);

    fclose(fin);
    fclose(fsym);
    fclose(fopt);
    fclose(flen);

    return 0;
}
```

```
saabith_kolakkattil@sk-VirtualBox:~$ cat optab.txt
LDA     00
STA     0C
ADD     18
SUB     1C
LDCH    50
STCH    54

saabith_kolakkattil@sk-VirtualBox:~$ cat symtab.txt
```

```
1012    FIVE
1015    ALPHA
1018    CHARZ
1019    C1
saabith_kolakkattil@sk-VirtualBox:~$ cat interm.txt
1000    COPY    START   1000
1000    -       LDA     FIVE
1003    -       STA     ALPHA
1006    -       LDCH    CHARZ
1009    -       STCH    C1
1012    FIVE    WORD    5
1015    ALPHA   RESW    1
1018    CHARZ   BYTE    C'Z'
1019    C1      RESB    1
1020    -       END     COPY
saabith_kolakkattil@sk-VirtualBox:~$ cat length.txt
20saabith_kolakkattil@sk-VirtualBox:~$ cc pass2.c
saabith_kolakkattil@sk-VirtualBox:~$ ./a.out
H^COPY^001000^000020
T^001000^001012^0C1015^501018^541019^000005^5A^
E^001000
```

ABSOLUTE LOADER

```
cat absolute.c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

void main()
{
    FILE *fp;
    int i, addr1, l, j, staddr1;
    char name[10], line[50], name1[10], addr[10], rec[10], ch, staddr[10];

    printf("enter program name:");
    scanf("%s", name);
    fp = fopen("objectcode.txt", "r");
    fscanf(fp, "%s", line);

    for (i = 2, j = 0;j < 6; i++, j++)
    {
        name1[j] = line[i];
    }
    name1[j] = '\0';
    printf("name from obj:%s\n", name1);

    if (strcmp(name, name1) == 0)
    {
        fscanf(fp, "%s", line);
        do
        {
            if (line[0] == 'T')
```

```
        {
            for (i = 2, j = 0;j < 6; i++, j++)
            {
                staddr[j] = line[i];
            }
            staddr[j] = '\0';
            staddr1 = atoi(staddr);
            i = 12;

            while (line[i] != '$')
            {
                if (line[i] != '^')
                {
                    printf("00%d\t%c%c\n", staddr1, line[i], line[i+1]);
                    staddr1++;
                    i = i + 2;
                }
                else i++;
            }
        }
        else if (line[0] == 'E')
        {
            printf("jump to execution address:%s", &line[2]);
        }

        fscanf(fp, "%s", line);
    } while (!feof(fp));
  }

  fclose(fp);
}
```

```
saabith_kolakkattil@sk-VirtualBox:~$ cat objectcode.txt
H^SAMPLE^001000^0035^
T^001000^0C^001003071009$
T^002000^03^111111$
E^001000

saabith_kolakkattil@sk-VirtualBox:~$ ./a.out
enter program name:SAMPLE
name from obj:SAMPLE
001000  00
001001  10
001002  03
001003  07
001004  10
001005  09
002000  11
002001  11
002002  11
jump to execution address:001000saabith_kolakkattil@sk-VirtualBox:~$
```

FCFS.c

```
#include<stdio.h>
void main()
{
    int i=0,j=0,b[10],c[20],p[20],w[20],t[20],a[20],n=0,m;
    float avgw=0,avgt=0;
    printf("Enter the number of process :");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        printf("Process ID :" );
        scanf("%d",&p[i]);
        printf("Arrival Time:");
        scanf("%d",&a[i]);
        printf("Burst Time :");
        scanf("%d",&b[i]);
    }
    printf("PID\tAT\tBT\n");
    for(i=0;i<n;i++)
    {
        printf("%d\t%d\t%d\n",p[i],a[i],b[i]);
    }
    int temp=0;
    for(i=0;i<n-1;i++)
    {
        for(j=0;j<n-1;j++)
        {
            if(a[j]>a[j+1])
            {
                temp=a[j];
                a[j]=a[j+1];
                a[j+1]=temp;
                temp=b[j];
                b[j]=b[j+1];
                b[j+1]=temp;
                temp=p[j];
                p[j]=p[j+1];
                p[j+1]=temp;
            }
        }
    }
    printf("PID\tAT\tBT\n");
    for(i=0;i<n;i++)
    {
        printf("%d\t%d\t%d\n",p[i],a[i],b[i]);
    }
```

```
    c[0]=a[0]+b[0];
    for(i=1;i<=n;i++)
    {
        if(a[i]<=c[i-1])
        {
            c[i]=c[i-1]+b[i];
        }
        else
        {
            c[i]=a[i]+b[i];
        }
    }
    for(i=0;i<n;i++)
    {
        t[i]=c[i]-a[i];
        w[i]=t[i]-b[i];
        avgw=avgw+w[i];
        avgt=avgt+t[i];
    }
    printf("\n\n\t\tcpu ALGORITHM");
    printf("\n\t\t\t GANTT CHART\n");
    printf("\n\t|-------------------------------------------|\n");
    printf("\t|");
    for(i=0;i<n;i++)
        printf(" %d |",p[i]);
    printf("\n\t|-------------------------------------------|\n");
    printf("\t");
    for(i=0;i<n;i++)
    {
        printf("   %d",c[i]);
    }
    printf("\n\t");
    avgw=avgw/n;
    avgt=avgt/n;
    printf("PID\tAT\tBT\tCT\tWT\tTT\n");
    for(i=0;i<n;i++)
    {
        printf("\t%d\t%d\t%d\t%d\t%d\t%d\n",p[i],a[i],b[i],c[i],w[i],t[i]);
    }
    printf("\nAverage Waiting Time: %f",avgw);
    printf("\nAverage Turnaround Time: %f ",avgt);
}
```

```
 cc Fcfs.c
[saabith@LabServer s4os]$ ./a.out
Enter the number of process :4
Process ID :5
Arrival Time:4
```

```
Burst Time :3
Process ID :2
Arrival Time:1
Burst Time :4
Process ID :3
Arrival Time:5
Burst Time :6
Process ID :7
Arrival Time:5
Burst Time :4
PID     AT      BT
5       4       3
2       1       4
3       5       6
7       5       4
PID     AT      BT
2       1       4
5       4       3
3       5       6
7       5       4


        cpu ALGORITHM
        GANTT CHART

        |-------------------------------------------|
        | 2 | 5 | 3 | 7 |
        |-------------------------------------------|
        5   8   14   18
        PID     AT      BT      CT      WT      TT
        2       1       4       5       0       4
        5       4       3       8       1       4
        3       5       6       14      3       9
        7       5       4       18      9       13

Average Waiting Time: 3.250000
Average Turnaround Time: 7.500000 [saabith@LabServer s4os]$
```

Shortest Job First

```
#include<stdio.h>
void main()
{
   int i=0,j=0,b[10],c[20],p[20],w[20],t[20],a[20],n=0,m,btt=0;
   float avgw=0,avgt=0;
   printf("Enter the number of process :");
   scanf("%d",&n);
   for(i=0;i<n;i++)
```

```
{
    printf("Process ID :" );
    scanf("%d",&p[i]);
    printf("Arrival Time:");
    scanf("%d",&a[i]);
    printf("Burst Time :");
    scanf("%d",&b[i]);
}
printf("PID\tAT\tBT\n");
for(i=0;i<n;i++)
{
    printf("%d\t%d\t%d\n",p[i],a[i],b[i]);
}
int temp=0;
for(i=0;i<n-1;i++)
{
    for(j=0;j<n-1;j++)
    {
        if(a[j]>a[j+1])
        {
            temp=a[j];
            a[j]=a[j+1];
            a[j+1]=temp;
            temp=b[j];
            b[j]=b[j+1];
            b[j+1]=temp;
            temp=p[j];
            p[j]=p[j+1];
            p[j+1]=temp;
        }
    }
}
btt=a[0];
for(i=0;i<n;i++)
{
    for(j=i+1;j<n;j++)
    {
        if(btt>=a[j] && b[j]<b[i])
        {
            temp=a[j];
            a[j]=a[i];
            a[i]=temp;
            temp=b[j];
            b[j]=b[i];
            b[i]=temp;
            temp=p[j];
            p[j]=p[i];
            p[i]=temp;
        }
    }
    btt+=b[i];
}
printf("afer bt \npid\tAT\tBT\t\n");
for(i=0;i<n;i++)
```

```
    {
        printf("%d\t%d\t%d\n",p[i],a[i],b[i]);
    }
    c[0]=a[0]+b[0];
    for(i=1;i<=n;i++)
    {
        if(a[i]<=c[i-1])
        {
            c[i]=c[i-1]+b[i];
        }
        else
        {
            c[i]=a[i]+b[i];
        }
    }
    for(i=0;i<n;i++)
    {
        t[i]=c[i]-a[i];
        w[i]=t[i]-b[i];
        avgw=avgw+w[i];
        avgt=avgt+t[i];
    }
    printf("\n\n\t\tcpu ALGORITHM");
    printf("\n\t\t\t GANTT CHART\n");
    printf("\n\t|-------------------------------------------|\n");
    printf("\t|");
    for(i=0;i<n;i++)
        printf(" %d |",p[i]);
    printf("\n\t|-------------------------------------------|\n");
    printf("\t");
    for(i=0;i<n;i++)
    {
        printf("   %d",c[i]);
    }
    printf("\n\t");
    avgw=avgw/n;
    avgt=avgt/n;
    printf("PID\tAT\tBT\tCT\tWT\tTT\n");
    for(i=0;i<n;i++)
    {
        printf("\t%d\t%d\t%d\t%d\t%d\t%d\n",p[i],a[i],b[i],c[i],w[i],t[i]);
    }
    printf("\nAverage Waiting Time: %f",avgw);
    printf("\nAverage Turnaround Time: %f ",avgt);
}
```

```
[saabith@LabServer s4os]$ cc Sjf.c
[saabith@LabServer s4os]$ ./a.out
Enter the number of process :4
Process ID :1
Arrival Time:5
Burst Time :1
Process ID :2
Arrival Time:2
Burst Time :5
Process ID :3
Arrival Time:1
Burst Time :3
Process ID :4
Arrival Time:6
Burst Time :2
PID     AT      BT
1       5       1
2       2       5
3       1       3
4       6       2
afer bt
pid     AT      BT
3       1       3
2       2       5
1       5       1
4       6       2
```

```
cpu ALGORITHM
GANTT CHART

|-------------------------------------------|
| 3 | 2 | 1 | 4 |
|-------------------------------------------|
4   9   10   12
PID     AT      BT      CT      WT      TT
3       1       3       4       0       3
2       2       5       9       2       7
1       5       1       10      4       5
4       6       2       12      4       6

Average Waiting Time: 2.500000
Average Turnaround Time: 5.250000 [saabith@LabServer s4os]$
```

// Priority

```
#include<stdio.h>
void main()
{
   int i=0,j=0,b[10],c[20],p[20],w[20],t[20],a[20],n=0,m,btt=0,pr[20];
   float avgw=0,avgt=0;
   printf("Enter the number of process :");
   scanf("%d",&n);
   for(i=0;i<n;i++)
   {
      printf("Process ID :" );
      scanf("%d",&p[i]);
      printf("Arrival Time:");
      scanf("%d",&a[i]);
      printf("Burst Time :");
      scanf("%d",&b[i]);
      printf("Process priority :" );
      scanf("%d",&pr[i]);
   }
   printf("PID\tAT\tBT\tPR\n");
   for(i=0;i<n;i++)
   {
      printf("%d\t%d\t%d\t%d\n",p[i],a[i],b[i],pr[i]);
   }
   int temp=0;
   for(i=0;i<n-1;i++)
   {
      for(j=0;j<n-1;j++)
      {
```

```
        if(a[j]>a[j+1])
        {
            temp=a[j];
            a[j]=a[j+1];
            a[j+1]=temp;
            temp=b[j];
            b[j]=b[j+1];
            b[j+1]=temp;
            temp=p[j];
            p[j]=p[j+1];
            p[j+1]=temp;
            temp=pr[j];
            pr[j]=pr[j+1];
            pr[j+1]=temp;
        }
    }
}
btt=a[0];
for(i=0;i<n;i++)
{
    for(j=i+1;j<n;j++)
    {
        if(btt>=a[j] && pr[j]<pr[i])
        {
            temp=a[j];
            a[j]=a[i];
            a[i]=temp;
            temp=b[j];
            b[j]=b[i];
            b[i]=temp;
            temp=p[j];
            p[j]=p[i];
            p[i]=temp;
            temp=pr[j];
            pr[j]=pr[i];
            pr[i]=temp;
        }
    }
    btt+=b[i];
}
printf("afer bt \npid\tAT\tBT\tPR\n");
for(i=0;i<n;i++)
{
    printf("%d\t%d\t%d\t%d\n",p[i],a[i],b[i],pr[i]);
}
c[0]=a[0]+b[0];
for(i=1;i<=n;i++)
{
    if(a[i]<=c[i-1])
    {
        c[i]=c[i-1]+b[i];
    }
    else
    {
```

```
            c[i]=a[i]+b[i];
        }
    }
    for(i=0;i<n;i++)
    {
        t[i]=c[i]-a[i];
        w[i]=t[i]-b[i];
        avgw=avgw+w[i];
        avgt=avgt+t[i];
    }
    printf("\n\n\t\tcpu ALGORITHM");
    printf("\n\t\t\t GANTT CHART\n");
    printf("\n\t|-------------------------------------------|\n");
    printf("\t|");
    for(i=0;i<n;i++)
        printf(" %d |",p[i]);
    printf("\n\t|-------------------------------------------|\n");
    printf("\t");
    for(i=0;i<n;i++)
    {
        printf("   %d",c[i]);
    }
    printf("\n\t");
    avgw=avgw/n;
    avgt=avgt/n;
    printf("PID\tAT\tBT\tPR\tCT\tWT\tTT\n");
    for(i=0;i<n;i++)
    {
        printf("\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n",p[i],a[i],b[i],pr[i],c[i],w[i],t[i]);
    }
    printf("\nAverage Waiting Time: %f",avgw);
    printf("\nAverage Turnaround Time: %f ",avgt);
}
```

```
[saabith@LabServer s4os]$ cc Priority.c
[saabith@LabServer s4os]$ ./a.out
Enter the number of process :4
Process ID :1
Arrival Time:4
Burst Time :3
Process priority :1
Process ID :2
Arrival Time:1
Burst Time :3
Process priority :6
Process ID :3
Arrival Time:1
Burst Time :4
Process priority :5
Process ID :5
Arrival Time:4
Burst Time :3
Process priority :8
PID     AT      BT      PR
1       4       3       1
2       1       3       6
3       1       4       5
5       4       3       8
afer bt
pid     AT      BT      PR
3       1       4       5
1       4       3       1
2       1       3       6
5       4       3       8


        cpu ALGORITHM
        GANTT CHART

        |-------------------------------------------|
        | 3 | 1 | 2 | 5 |
        |-------------------------------------------|
        5   8   11   14
        PID     AT      BT      PR      CT      WT      TT
        3       1       4       5       5       0       4
        1       4       3       1       8       1       4
        2       1       3       6       11      7       10
        5       4       3       8       14      7       10

Average Waiting Time: 3.750000
```

Average Turnaround Time: 7.000000 [saabith@LabServer s4os]$

Round Robin

```
#include<stdio.h>
void main()
{
    int p[10],a[10],b[10],c[10],t[10],w[10],rt[10],q,time=0,n,i,count=0;
    float avgw=0,avgt=0;
    printf("Enter the no of process:");
    scanf("%d",&n);
    printf("Enter the Quantum time:");
    scanf("%d",&q);
    printf("Enter the details of processor\n");
    for(i=0;i<n;i++)
    {
        printf("Process ID :" );
        scanf("%d",&p[i]);
        printf("Burst Time :");
        scanf("%d",&b[i]);
        rt[i]=b[i];
    }
    printf("input details are\n");
    printf("all process are arrived at time 0\n");
    printf("\tPID\tBT\t\n");
    for(i=0;i<n;i++)
    {
        printf("\t%d\t%d\n",p[i],b[i]);
    }
    count=n;
    while(count>0)
    {
        for(i=0;i<n;i++)
        {
            if(rt[i]>0)
            {
                if(rt[i]>q)
                {
                    rt[i]=rt[i]-q;
                    time=time+q;
                }
                else
                {
                    time=time+rt[i];
                    rt[i]=0;
                    c[i]=time;
                    count--;
                }
            }
        }
    }
    for(i=0;i<n;i++)
```

```
    {
        t[i]=c[i];
        w[i]=t[i]-b[i];
        avgw=avgw+w[i];
        avgt=avgt+t[i];
    }
    printf("\n\t\t\t GANTT CHART\n");
    printf("\n\t|-------------------------------------------|\n");
    printf("\t|");
    for(i=0;i<n;i++)
        printf(" %d |",p[i]);
    printf("\n\t|-------------------------------------------|\n");
    printf("\t");
    for(i=0;i<n;i++)
    {
        printf("   %d",c[i]);
    }
    printf("\n\t");
    avgw=avgw/n;
    avgt=avgt/n;
    printf("PID\tBT\tCT\tWT\tTT\n");
    for(i=0;i<n;i++)
    {
        printf("\t%d\t%d\t%d\t%d\t%d\n",p[i],b[i],c[i],w[i],t[i]);
    }
    printf("\nAverage Waiting Time: %f",avgw);
    printf("\nAverage Turnaround Time: %f ",avgt);
}
```

```
 [saabith@LabServer s4os]$ vi Round_robin.c
[saabith@LabServer s4os]$ cc Round_robin.c
[saabith@LabServer s4os]$ ./a.out
Enter the no of process:4
Enter the Quantum time:2
Enter the details of processor
Process ID :1
Burst Time :4
Process ID :2
Burst Time :5
Process ID :3
Burst Time :1
Process ID :5
Burst Time :3
input details are
all process are arrived at time 0
        PID     BT
        1       4
        2       5
        3       1
        5       3

        GANTT CHART
```

```
|-------------------------------------------|
| 1 | 2 | 3 | 5 |
|-------------------------------------------|
9   13   5   12
PID     BT      CT      WT      TT
1       4       9       5       9
2       5       13      8       13
3       1       5       4       5
5       3       12      9       12

Average Waiting Time: 6.500000
Average Turnaround Time: 9.750000 [saabith@LabServer s4os]$
```

Page replacement

Fifo

```
#include <stdio.h>
void main()
{
	int i,j,n,A[50],frame[20],no,k,avail,count=0;
	printf("Enter the number of pages\n");
	scanf("%d",&n);
```

```
	printf("Enter the page number\n");
	for(i=1;i<=n;i++)
	{
		scanf("%d",&A[i]);
	}
	printf("Enter the number of frames\n");
	scanf("%d",&no);
	for(i=0;i<no;i++)
	{
		frame[i]=-1;
		j=0;
	}
	printf("REFERENCE STRING\t PAGE FRAMES\n");
	for(i=1;i<=n;i++)
	{
		printf("%d\t\t",A[i]);
		avail=0;
		for(k=0;k<no;k++)
			if(frame[k]==A[i])
			avail=1;
			if(avail==0)
			{
			frame[j]=A[i];
			j=(j+1)%no;
			count++;
			for(k=0;k<no;k++)
			printf("%d\t", frame[k]);
			}
	printf("\n");
	}
		printf("Page Fault is %d.\n",count);
}
```

```
[saabith@LabServer s4os]$ vi pfifo.c
[saabith@LabServer s4os]$ cc pfifo.c
[saabith@LabServer s4os]$ ./a.out
Enter the number of pages
7
Enter the page number
3
4
5
6
5
7
6
Enter the number of frames
3
```

```
REFERENCE STRING  PAGE FRAMES
3         3         -1        -1
4         3         4         -1
5         3         4         5
6         6         4         5
5
7         6         7         5
6
Page Fault is 5.
[saabith@LabServer s4os]$
```

LRU.c

```
#include <stdio.h>

void main() {
        int i, j, n, A[20], frame[20], no, k, avail, count = 0;
        int time[20];
        int counter = 0;

        printf("Enter the number of pages\n");
        scanf("%d", &n);

        printf("Enter the page numbers\n");
        for(i = 1; i <= n; i++) {
        scanf("%d", &A[i]);
        }

        printf("Enter the number of frames\n");
        scanf("%d", &no);

        for(i = 0; i < no; i++) {
        frame[i] = -1;
        time[i] = -1;
        }

        printf("REFERENCE STRING\t PAGE FRAMES\n");

        for(i = 1; i <= n; i++) {
        printf("%d\t\t", A[i]);
        avail = 0;

        for(k = 0; k < no; k++) {
        if(frame[k] == A[i]) {
        avail = 1;
        time[k] = counter;
        break;
        }
        }

        if(avail == 0) {
```

```
        int lru_index = 0;
        for(k = 1; k < no; k++) {
        if(time[k] < time[lru_index]) {
                lru_index = k;
        }
        }

        frame[lru_index] = A[i];
        time[lru_index] = counter;
        count++;
        }

        for(k = 0; k < no; k++) {
        if(frame[k] == -1)
        printf("-1 ");
        else
        printf("%d ", frame[k]);
        }
        printf("\n");

        counter++;
        }

        printf("Total Page Faults: %d\n", count);
}
```

OUTPUT

```
cc plru.c
[saabith@LabServer s4os]$ ./a.out
Enter the number of pages
7
Enter the page numbers
5
4
5
7
8
7
6
Enter the number of frames
3
REFERENCE STRING  PAGE FRAMES
5       5       -1      -1
4       5       4       -1
5       5       4       -1
7       5       4       7
```

8	5	8	7
7	5	8	7
6	6	8	7

Page Faults: 5.

DISK_SCHEDUL_FCFS

```
[saabith@LabServer s4os]$ cat dfcfs.c
#include<stdio.h>
#include<stdlib.h>
int main(){
int ReadyQueue[100],i,n, TotalHeadMov=0,initial;
printf("Enter the number of requests: ");
scanf("%d", &n);
printf("Enter the sequence of request:\n");
for(i=0;i<n;i++){
scanf("%d", &ReadyQueue[i]);
}
printf("Enter initial head position: ");
scanf("%d", &initial);
for(i=0;i<n;i++){
TotalHeadMov=TotalHeadMov+abs(ReadyQueue[i]-initial);
initial=ReadyQueue[i];
}
printf("Total Head Movement=%d", TotalHeadMov);
printf("\n");
}
```

OUTPUT

```
[saabith@LabServer s4os]$ ./a.out
Enter the number of requests: 7
Enter the sequence of request:
82
170
43
140
```

```
24
16
190
Enter initial head position: 50

Total Head Movement=642
```

DISk SCHEDULING SCan

```
[saabith@LabServer s4os]$ cat dscan.c
include<stdio.h>
#include<stdlib.h>

void main() {
	int ReadyQueue[100], i, j, n, TotalHeadMov = 0, initial, diskSize, direction, temp;

	printf("Enter the number of requests: ");
	scanf("%d", &n);

	printf("Enter the sequence of requests:\n");
	for(i = 0; i < n; i++) {
	scanf("%d", &ReadyQueue[i]);
	}

	printf("Enter the initial head position: ");
	scanf("%d", &initial);

	printf("Enter the disk size (max value): ");
	scanf("%d", &diskSize);

	printf("Enter the direction of the head (0 for left, 1 for right): ");
	scanf("%d", &direction);

	for(i = 0; i < n-1; i++) {
	for(j = i+1; j < n; j++) {
	if(ReadyQueue[i] > ReadyQueue[j]) {
		temp = ReadyQueue[i];
		ReadyQueue[i] = ReadyQueue[j];
		ReadyQueue[j] = temp;
	}
	}
	}


	printf("SCAN Scheduling:\n");

	if(direction == 1) {
```

```
        for(i = 0; i < n; i++) {
        if(ReadyQueue[i] >= initial) {
                break;
        }
        }

        for(j = i; j < n; j++) {
        TotalHeadMov += abs(ReadyQueue[j] - initial);
        initial = ReadyQueue[j];
        }
        TotalHeadMov += abs(diskSize - initial-1);
        initial= diskSize-1;

        for(j = i - 1; j >= 0; j--) {
        TotalHeadMov += abs(ReadyQueue[j] - initial);
        initial = ReadyQueue[j];
        }
        } else {

        for(i = n-1; i >= 0; i--) {
        if(ReadyQueue[i] <= initial) {
                break;
        }
        }

        for(j = i; j >= 0; j--) {
        TotalHeadMov += abs(ReadyQueue[j] - initial);
        initial = ReadyQueue[j];
        }

        TotalHeadMov += initial;
                initial= 0;


        for(j = i + 1; j < n; j++) {
        TotalHeadMov += abs(ReadyQueue[j] - initial);
        initial = ReadyQueue[j];
        }
        }

        printf("Total Head Movement = %d\n", TotalHeadMov);
}
```

```
[saabith@LabServer s4os]$ ./a.out
```

```
Enter the number of requests: 8
Enter the sequence of requests:
95
180
34
119
11
123
62
64
Enter the initial head position: 50
Enter the disk size (max value): 200
Enter the direction of the head (0 for left, 1 for right): 0
SCAN Scheduling:
Total Head Movement = 230
```

CSCAN

```
#include<stdio.h>
#include<stdlib.h>

int main() {
  int RQ[100], i, j, n, TotalHeadMovement = 0, initial, size, move;
  printf("Enter the number of requests: ");
  scanf("%d", & n);
  printf("Enter the requests sequence:\n");
  for (i = 0; i < n; i++)
          scanf("%d", & RQ[i]);
  printf("Enter initial head position: ");
  scanf("%d", & initial);
  printf("Enter total disk size: ");
  scanf("%d", & size);
  printf("Enter the head movement direction for high 1 and for low 0: ");
  scanf("%d", & move);
  for (i = 0; i < n; i++) {
          for (j = 0; j < n; j++) {
          if (RQ[j] > RQ[j + 1]) {
          int temp;
          temp = RQ[j];
          RQ[j] = RQ[j + 1];
          RQ[j + 1] = temp;
          }
          }
  }
  int index;
```

```
  for (i = 0; i < n; i++) {
        if (initial < RQ[i]) {
        index = i;
        break;
        }
  }
  if (move == 1) {
        for (i = index; i < n; i++) {
        TotalHeadMovement = TotalHeadMovement + abs(RQ[i] - initial);
        initial = RQ[i];
        }
        TotalHeadMovement = TotalHeadMovement + abs(size - RQ[i - 1] - 1);
        TotalHeadMovement = TotalHeadMovement + abs(size - 1 - 0);
        initial = 0;
        for (i = 0; i < index; i++) {
        TotalHeadMovement = TotalHeadMovement + abs(RQ[i] - initial);

        initial = RQ[i];
        }
  } else {
        for (i = index - 1; i >= 0; i--) {
        TotalHeadMovement = TotalHeadMovement + abs(RQ[i] - initial);
        initial = RQ[i];
        }
        TotalHeadMovement = TotalHeadMovement + abs(RQ[i + 1] - 0);
        TotalHeadMovement = TotalHeadMovement + abs(size - 1 - 0);
        initial = size - 1;
        for (i = n - 1; i >= index; i--) {
        TotalHeadMovement = TotalHeadMovement + abs(RQ[i] - initial);
        initial = RQ[i];
        }
  }
  printf("Total head movement is %d", TotalHeadMovement);
  printf("\n");
  return 0;
}
```